Here are some examples to clarify the expected behavior of your application for each subtask.

Example 1: Subtask I - Basic "check" and "borrow" Actions

This example demonstrates the fundamental communication flow between a single student and the library server.

Scenario: The server has a book `BOOK0001` with 3 copies and reservation code `XYZ123`.

Part A: Checking for a Book

- **Student Client Input:**

```
Enter action (check/borrow/quit): check
Enter book ID: BOOK0001
```

- **Data Sent to Server:** `b'check:BOOK0001:'`
- **Library Server Log:**

```
New student connected from ('127.0.0.1', 54321)
Student ('127.0.0.1', 54321) - Action: check, Book: BOOK0001
  Response: available:3
```

- **Student Client Output:**

```
Sent: 'check:BOOK0001:'
Response: available:3
✓ Book is available! Copies: 3
```

Part B: Successfully Borrowing a Book

- **Student Client Input:**

```
Enter action (check/borrow/quit): borrow
Enter book ID: BOOK0001
Enter reservation code: XYZ123
```

- **Data Sent to Server:** `b'borrow:BOOK0001:XYZ123'`
- **Library Server Log:**

```
Student ('127.0.0.1', 54321) - Action: borrow, Book: BOOK0001
  SUCCESS! Book borrowed. Remaining copies: 2
  Response: success
```

- **Student Client Output:**

```
Sent: 'borrow:BOOK0001:XYZ123'
Response: success
✓ Book borrowed successfully!
```

- **Explanation:** The server verified the book ID and reservation code, confirmed copies were available, decremented the count from 3 to 2, and sent a `success` message.

Example 2: Subtask II - Server Startup Options

This example shows how the library server should behave based on its command-line arguments.

Option A: Loading from a `books.json` file

- **Command:**

```
python3 library_server.py 127.0.0.1 5000 books.json 127.0.0.1 6000
```

- **Expected Server Log Output:**

```
Loading books from books.json...
Library inventory:
  BOOK0001: Network Programming - 3 copies (code: aB1cD2)
  BOOK0002: Data Structures - 5 copies (code: eF3gH4)

Library server listening on 127.0.0.1:5000
...
```

- **Explanation:** The server reads the specified JSON file and generates a random reservation code for each book loaded.

Option B: Generating Random Books

- **Command:**

```
python3 library_server_random.py 127.0.0.1 5000 2 127.0.0.1 6000
```

- **Expected Server Log Output:**

```
Generating 2 random books...
Library inventory:
  A1B2C3D4: Algorithms - 7 copies (code: xY1zP2)
  E5F6G7H8: Databases - 2 copies (code: qW3eR4)

Library server listening on 127.0.0.1:5000
...
```

- **Explanation:** The server generates 2 random book entries as specified by the command-line argument.

Example 3: Subtask III - Concurrent Connections

This example demonstrates how the server handles multiple students using `select`.

- **Timeline:**
  i. **Student A** connects. The server accepts the connection.
  ii. **Student B** connects. The server accepts this second connection **without** waiting for Student A to finish.
  iii. **Student A** sends a `check` request and receives a response.
  iv. **Student B** sends a `borrow` request and receives a response.
- **Expected Library Server Log:**

```
Library server listening on 127.0.0.1:5000
...
New student connected from ('127.0.0.1', 54321)  <-- Student A connects

New student connected from ('127.0.0.1', 54322)  <-- Student B connects

Student ('127.0.0.1', 54321) - Action: check, Book: BOOK0001 <-- Student A's request
  Response: available:3

Student ('127.0.0.1', 54322) - Action: borrow, Book: BOOK0002 <-- Student B's request
  SUCCESS! Book borrowed. Remaining copies: 4
  Response: success
```

- **Explanation:** The server is able to interleave its handling of Student A and Student B, proving it is not blocking on a single client.

Example 4: Subtask IV - UDP Notification System

This example shows the interaction between all three components during a successful borrow operation.

- **Scenario:** A student borrows book `A4KG6QYL`. Before the borrow, it had 4 copies.
- **Terminals:**

Student Client Terminal	Library Server Log	Notification Server Log

`Enter action: borrow`
`Enter book ID: A4KG6QYL`
`Enter reservation code: 3TTP43`
	`Student ... – Action: borrow, Book: A4KG6QYL`	*(Waiting...)*

(Waiting for response...)	`SUCCESS! Book borrowed. Remaining copies: 3`	`Received notification from ...`
		`Time: 2025-11-28...`
		`Student: 127.0.0.1`
		`Book: A4KG6QYL`
		`Remaining: 3 copies`

(Waiting for response...)	*(Waiting for ACK...)*	`✓ Logged to borrowing_log.json`
(Waiting for response...)	`Notification acknowledged: ACK` `Response: success`	*(Waiting...)*
`Response: success` `✓ Book borrowed successfully!`	*(Ready for next action)*	*(Ready for next notification)*

- **Explanation:** The Library Server successfully processed the borrow, sent a UDP notification, waited for the `ACK`, and only then sent the final `success` confirmation back to the student client.

Bonus Subtask Example

- **Explanation:** If you implement the bonus, the application's logic and behavior will be identical to the examples above. The only difference is the data sent over the network. Instead of a human-readable string like `b'borrow:BOOK0001:XYZ123'`, the client would send a fixed-size block of binary data created with `struct.pack()`, and the server would parse it with `struct.unpack()`.